Practice Exercise: CC-UMA vs CC-NUMA in Sniper using pthread Matrix–Vector Multiplication (4-Core Experiments)

Parallel and Distributed Computing (Spring 2026)

This exercise is designed to build practical intuition for how shared-memory program performance depends on the underlying architecture: the same pthreaded matrix–vector multiplication can behave very differently on a uniform-memory (CC-UMA) versus non-uniform-memory (CC-NUMA) system because memory locality, contention, and interconnect distance directly influence latency and bandwidth. By recreating UMA, NUMA, and "remote-memory" scenarios in Sniper, you will learn to treat performance as a debugging problem: form hypotheses (e.g., "remote accesses are dominating"), validate them using profiling statistics (`sim.out` and `dumpstats`), and then apply targeted optimizations (thread pinning and data first-touch / locality-aware initialization) to improve performance on a specific architecture.

Overview

This exercise uses the Sniper multicore simulator to compare:

1. a 4-core **CC-UMA** machine,
2. a 4-core **CC-NUMA** machine,
3. the same 4-core CC-NUMA machine under a configuration that amplifies the **remote memory effect**,
4. and finally a **data-locality optimized** program variant on the same CC-NUMA (remote-effect) configuration.

Prerequisites

- Environment: **Ubuntu 20.04 LTS VM** (recommended for Sniper 7.4 to avoid Python/-toolchain incompatibilities on newer Ubuntu versions). The ubuntu image tested for this excercise is downloaded from: https://www.linuxvmimages.com/images/ubuntu-2004/
- Packages: build tools, Git/Wget, **Python 2** (required by Sniper 7.4 build scripts), and `time`.
- Basic knowledge of pthreads and command-line usage.

1 Step 1: Install and Verify Sniper (Ubuntu 20.04 VM)

1.1 Verify OS version (must be Ubuntu 20.04)

```
lsb_release -a
```

1.2 Install required packages (Python 2 required)

```
sudo apt update

sudo apt install -y python3 python3-dev python3-distutils

sudo apt install -y zlib1g-dev libbz2-dev libsqlite3-dev libboost-all-dev

sudo apt install -y wget git build-essential make
```

Verify Python:

```
python --version
```

1.3 Create `pdc-labs` folder and obtain Sniper package

Use the provided Sniper archive on GCR (`sniper-latest.tgz`).

```
mkdir -p ~/pdc-labs && cd ~/pdc-labs

# Copy sniper-latest.tgz into ~/pdc-labs (from GCR) then:
tar -xzf sniper-latest.tgz

# Rename extracted folder to snipersim (folder name may differ; adjust if needed)
mv sniper-7.4 snipersim
```

1.4 Build Sniper

```
cd ~/pdc-labs/snipersim
make clean
make -j"$(nproc)" USE_PIN=1 USE_SDE=
```

1.5 Verify Sniper using built-in test

```
cd test/fft
make run
```

Create a tmp folder and Export TMPDIR environment variable (if you close terminal and reopen you would be required to Export the TMPDIR again.

```
mkdir -p ~/pdc-labs/tmp
export TMPDIR=~/pdc-labs/tmp
```

Return to Sniper root:

```
cd ~/pdc-labs/snipersim
```

2 Step 2: Build the pthread MatVec Program (Baseline)

2.1 Create a working directory

```
mkdir -p ~/pdc-labs/lab-cc-numa-matvec
cd ~/pdc-labs/lab-cc-numa-matvec
```

2.2 Create `matvec_pthreads.c`

Create the file `matvec_pthreads.c` with the following content.

```
#define _GNU_SOURCE
#include <pthread.h>
#include <sched.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <time.h>
#include <unistd.h>

typedef struct {
  int tid;
  int nthreads;
  int N; // rows
  int M; // cols
  int R; // repetitions
  double *A; // N x M row-major
  double *x; // M
  double *y; // N
  int pin_mode; // 0=none, 1=compact, 2=remote-stress
} worker_args_t;

static inline double now_sec() {
  struct timespec ts;
  clock_gettime(CLOCK_MONOTONIC, &ts);
  return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
}

static void pin_to_cpu(int cpu) {
  cpu_set_t set;
  CPU_ZERO(&set);
  CPU_SET(cpu, &set);
  if (sched_setaffinity(0, sizeof(set), &set) != 0) perror("sched_setaffinity");
}

void *worker(void *arg) {
  worker_args_t *w = (worker_args_t*)arg;
  int ncpu = (int)sysconf(_SC_NPROCESSORS_ONLN);

  // pin_mode:
  // 0: no pinning
  // 1: compact (tid -> cpu tid)
  // 2: remote-stress (tid -> cpu tid + 4) to push threads away (assumes >= 8 CPUs
      available)
  if (w->pin_mode == 1) pin_to_cpu(w->tid % ncpu);
  else if (w->pin_mode == 2) pin_to_cpu((w->tid + 4) % ncpu);

  int start = (w->tid * w->N) / w->nthreads;
  int end = ((w->tid + 1) * w->N) / w->nthreads;

  for (int r = 0; r < w->R; r++) {
    for (int i = start; i < end; i++) {
      const double *row = &w->A[(size_t)i * (size_t)w->M];
      double sum = 0.0;
      for (int j = 0; j < w->M; j++) sum += row[j] * w->x[j];
      w->y[i] = sum;
```

```
    }
  }
  return NULL;
}

int main(int argc, char **argv) {
  if (argc < 6) {
    fprintf(stderr, "Usage: %s N M R nthreads pin_mode(0/1/2)\n", argv[0]);
    fprintf(stderr, "Example: %s 4096 4096 2 4 1\n", argv[0]);
    return 1;
  }

  int N = atoi(argv[1]);
  int M = atoi(argv[2]);
  int R = atoi(argv[3]);
  int nthreads = atoi(argv[4]);
  int pin_mode = atoi(argv[5]);

  double *A = NULL, *x = NULL, *y = NULL;
  if (posix_memalign((void**)&A, 64, (size_t)N * (size_t)M * sizeof(double)) != 0)
      return 2;
  if (posix_memalign((void**)&x, 64, (size_t)M * sizeof(double)) != 0) return 2;
  if (posix_memalign((void**)&y, 64, (size_t)N * sizeof(double)) != 0) return 2;

  for (int j = 0; j < M; j++) x[j] = (double)((j % 100) - 50) * 0.01;
  for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) {
      A[(size_t)i*(size_t)M + (size_t)j] = (double)((i + j) % 100) * 0.001;
    }
    y[i] = 0.0;
  }

  pthread_t *threads = (pthread_t*)malloc((size_t)nthreads * sizeof(pthread_t));
  worker_args_t *args = (worker_args_t*)malloc((size_t)nthreads * sizeof(
      worker_args_t));

  double t0 = now_sec();
  for (int t = 0; t < nthreads; t++) {
    args[t] = (worker_args_t){ .tid=t, .nthreads=nthreads, .N=N, .M=M, .R=R,
                               .A=A, .x=x, .y=y, .pin_mode=pin_mode };
    if (pthread_create(&threads[t], NULL, worker, &args[t]) != 0) {
      perror("pthread_create");
      return 3;
    }
  }
  for (int t = 0; t < nthreads; t++) pthread_join(threads[t], NULL);
  double t1 = now_sec();

  double chk = 0.0;
  for (int i = 0; i < N; i += (N/16 + 1)) chk += y[i];

  printf("DONE N=%d M=%d R=%d threads=%d pin_mode=%d time=%.6f sec checksum=%.6f\n",
         N, M, R, nthreads, pin_mode, (t1 - t0), chk);

  free(threads);
  free(args);
  free(A); free(x); free(y);
  return 0;
```

```
}
```

2.3 Compile (baseline)

```
gcc -O3 -march=native -pthread -o matvec_baseline matvec_pthreads.c
```

2.4 Place binaries in a stable location

```
mkdir -p ~/pdc-labs/bin
cp matvec_baseline ~/pdc-labs/bin/
```

3 Step 3: Sniper Experiment 1, a 4-Core CC-UMA Configuration

3.1 Create UMA config file

```
cd ~/pdc-labs/snipersim
cp config/gainestown.cfg config/lab_4c_uma.cfg
```

Edit `config/lab_4c_uma.cfg` and set:

```
# lab_4c_uma.cfg
# 4-core CC-UMA: single DRAM controller (uniform access)

#include gainestown

[perf_model/dram]
num_controllers = 1
controller_positions = "0"
controllers_interleaving = 0

# Keep the baseline Gainestown DRAM timing/bandwidth (override only if you need)
latency = 45
per_controller_bandwidth = 7.6
chips_per_dimm = 8
dimms_per_controller = 4
```

3.2 Run the program on UMA (lab_4c_uma.cfg)

Use a workload that is large enough to be memory-relevant but still finishes quickly.

```
mkdir -p ~/pdc-labs/results/exp1_uma
./run-sniper -c lab_4c_uma -n 4 -d ~/pdc-labs/results/exp1_uma -- \
  ~/pdc-labs/bin/matvec_baseline 4096 4096 2 4 1
```

3.3 Evaluate performance and profiling stats

```
cat ~/pdc-labs/results/exp1_uma/sim.out | head -n 80
./tools/dumpstats.py -d ~/pdc-labs/results/exp1_uma > ~/pdc-labs/results/exp1_uma.
    stats.txt
head -n 80 ~/pdc-labs/results/exp1_uma.stats.txt
```

Record at minimum:

- runtime (from `sim.out`)
- IPC (from `sim.out`)
- a small set of memory-related stats (DRAM activity / cache misses) from `dumpstats.py`

4 Step 4: Sniper Experiment 2, a 4-Core CC-NUMA Configuration

4.1 Create NUMA config file (2 controllers)

```
cd ~/pdc-labs/snipersim
cp config/lab_4c_uma.cfg config/lab_4c_numa.cfg
```

Edit `config/lab_4c_numa.cfg` and set:

```
# lab_4c_numa.cfg
# 4-core CC-NUMA: two DRAM controllers (non-uniform access)

#include gainestown

[perf_model/dram]
num_controllers = 2
controller_positions = "0,2"
controllers_interleaving = 0

latency = 45
per_controller_bandwidth = 7.6
chips_per_dimm = 8
dimms_per_controller = 4
```

4.2 Run the Program on NUMA(lab_4c_numa.cfg)

```
mkdir -p ~/pdc-labs/results/exp2_numa
./run-sniper -c lab_4c_numa -n 4 -d ~/pdc-labs/results/exp2_numa -- \
  ~/pdc-labs/bin/matvec_baseline 4096 4096 2 4 1
```

4.3 Evaluate performance and profiling stats

```
cat ~/pdc-labs/results/exp2_numa/sim.out | head -n 80
./tools/dumpstats.py -d ~/pdc-labs/results/exp2_numa > ~/pdc-labs/results/exp2_numa.
    stats.txt
head -n 80 ~/pdc-labs/results/exp2_numa.stats.txt
```

5 Step 5: Sniper Experiment 3, a 4-Core CC-NUMA with Remote Memory Effect

5.1 Create remote-effect NUMA config

We amplify the "remote memory effect" by:

- keeping two memory controllers, and
- placing them such that certain cores are topologically farther from one controller.

Copy config:

```
cd ~/pdc-labs/snipersim
cp config/lab_4c_numa.cfg config/lab_4c_numa_remote.cfg
```

Edit `config/lab_4c_numa_remote.cfg` and set:

```
# lab_4c_numa_remote.cfg
# 4-core CC-NUMA with amplified remote effect:
# place controllers farther apart topologically

#include gainestown

[perf_model/dram]
num_controllers = 2
controller_positions = "0,3"
controllers_interleaving = 0

latency = 45
per_controller_bandwidth = 7.6
chips_per_dimm = 8
dimms_per_controller = 4
```

5.2 Run baseline program using remote-stress pinning

Use `pin_mode=2` to push threads away from their compact placement.

```
mkdir -p ~/pdc-labs/results/exp3_numa_remote
./run-sniper -c lab_4c_numa_remote -n 4 -d ~/pdc-labs/results/exp3_numa_remote -- \
  ~/pdc-labs/bin/matvec_baseline 4096 4096 2 4 2
```

5.3 Evaluate performance and profiling stats

```
cat ~/pdc-labs/results/exp3_numa_remote/sim.out | head -n 80
./tools/dumpstats.py -d ~/pdc-labs/results/exp3_numa_remote > ~/pdc-labs/results/
    exp3_numa_remote.stats.txt
head -n 80 ~/pdc-labs/results/exp3_numa_remote.stats.txt
```

6 Step 6: Experiment 4 — Data Locality Optimized Variant on the Same Remote-NUMA Config

6.1 Create a locality-optimized variant

The goal is to encourage better locality by:

- touching/initializing the matrix in the same thread partitioning scheme as computation (first-touch style),
- using `pin_mode=1` (compact) so each thread repeatedly works on its own chunk.

Create `matvec_locality.c`:

```
#define _GNU_SOURCE
#include <pthread.h>
#include <sched.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <time.h>
#include <unistd.h>

typedef struct {
  int tid, nthreads, N, M, R;
  double *A, *x, *y;
  int pin_mode;
} args_t;

static inline double now_sec() {
  struct timespec ts; clock_gettime(CLOCK_MONOTONIC, &ts);
  return (double)ts.tv_sec + 1e-9 * (double)ts.tv_nsec;
}

static void pin_to_cpu(int cpu) {
  cpu_set_t set; CPU_ZERO(&set); CPU_SET(cpu, &set);
  if (sched_setaffinity(0, sizeof(set), &set) != 0) perror("sched_setaffinity");
}

static inline void do_pinning(int tid, int pin_mode) {
  int ncpu = (int)sysconf(_SC_NPROCESSORS_ONLN);
  if (pin_mode == 1) pin_to_cpu(tid % ncpu);
  else if (pin_mode == 2) pin_to_cpu((tid + 4) % ncpu);
}

void *init_worker(void *p) {
  args_t *a = (args_t*)p;
  do_pinning(a->tid, a->pin_mode);
  int start = (a->tid * a->N) / a->nthreads;
  int end = ((a->tid + 1) * a->N) / a->nthreads;

  // first-touch initialization by the same threads that will compute
  for (int i = start; i < end; i++) {
    for (int j = 0; j < a->M; j++) {
      a->A[(size_t)i*(size_t)a->M + (size_t)j] = (double)((i + j) % 100) * 0.001;
    }
    a->y[i] = 0.0;
  }
  return NULL;
}

void *compute_worker(void *p) {
  args_t *a = (args_t*)p;
  do_pinning(a->tid, a->pin_mode);
  int start = (a->tid * a->N) / a->nthreads;
  int end = ((a->tid + 1) * a->N) / a->nthreads;

  for (int r = 0; r < a->R; r++) {
    for (int i = start; i < end; i++) {
      const double *row = &a->A[(size_t)i * (size_t)a->M];
      double sum = 0.0;
```

```
      for (int j = 0; j < a->M; j++) sum += row[j] * a->x[j];
      a->y[i] = sum;
    }
  }
  return NULL;
}

int main(int argc, char **argv) {
  if (argc < 6) {
    fprintf(stderr, "Usage: %s N M R nthreads pin_mode(0/1/2)\n", argv[0]);
    return 1;
  }
  int N = atoi(argv[1]), M = atoi(argv[2]), R = atoi(argv[3]);
  int nthreads = atoi(argv[4]);
  int pin_mode = atoi(argv[5]);

  double *A=NULL,*x=NULL,*y=NULL;
  if (posix_memalign((void**)&A, 64, (size_t)N*(size_t)M*sizeof(double)) != 0)
      return 2;
  if (posix_memalign((void**)&x, 64, (size_t)M*sizeof(double)) != 0) return 2;
  if (posix_memalign((void**)&y, 64, (size_t)N*sizeof(double)) != 0) return 2;

  for (int j = 0; j < M; j++) x[j] = (double)((j % 100) - 50) * 0.01;

  pthread_t *th = (pthread_t*)malloc((size_t)nthreads*sizeof(pthread_t));
  args_t *args = (args_t*)malloc((size_t)nthreads*sizeof(args_t));

  // Parallel init (first-touch)
  for (int t=0;t<nthreads;t++) {
    args[t] = (args_t){ .tid=t,.nthreads=nthreads,.N=N,.M=M,.R=R,.A=A,.x=x,.y=y,.
        pin_mode=pin_mode };
    pthread_create(&th[t], NULL, init_worker, &args[t]);
  }
  for (int t=0;t<nthreads;t++) pthread_join(th[t], NULL);

  // Compute
  double t0 = now_sec();
  for (int t=0;t<nthreads;t++) pthread_create(&th[t], NULL, compute_worker, &args[t
      ]);
  for (int t=0;t<nthreads;t++) pthread_join(th[t], NULL);
  double t1 = now_sec();

  double chk = 0.0;
  for (int i = 0; i < N; i += (N/16 + 1)) chk += y[i];

  printf("LOCALITY DONE N=%d M=%d R=%d threads=%d pin_mode=%d time=%.6f sec checksum
      =%.6f\n",
         N,M,R,nthreads,pin_mode,(t1-t0),chk);

  free(th); free(args); free(A); free(x); free(y);
  return 0;
}
```

6.2 Compile and copy binary

```
cd ~/pdc-labs/lab-cc-numa-matvec
gcc -O3 -march=native -pthread -o matvec_locality matvec_locality.c
```

```
cp matvec_locality ~/pdc-labs/bin/
```

6.3 Run locality-optimized program on the same remote-NUMA config

Use `pin_mode=1` (compact) to maintain stable per-thread locality.

```
mkdir -p ~/pdc-labs/results/exp4_locality_on_remote_numa
cd ~/pdc-labs/snipersim
./run-sniper -c lab_4c_numa_remote -n 4 -d ~/pdc-labs/results/
    exp4_locality_on_remote_numa -- \
  ~/pdc-labs/bin/matvec_locality 4096 4096 2 4 1
```

6.4 Evaluate performance and profiling stats

```
cat ~/pdc-labs/results/exp4_locality_on_remote_numa/sim.out | head -n 80
./tools/dumpstats.py -d ~/pdc-labs/results/exp4_locality_on_remote_numa \
  > ~/pdc-labs/results/exp4_locality_on_remote_numa.stats.txt
head -n 80 ~/pdc-labs/results/exp4_locality_on_remote_numa.stats.txt
```

What to observe in different variants?s

UMA vs NUMA vs Remote-NUMA vs Locality-Optimized

Include a table with:

- Experiment name (Exp1 UMA, Exp2 NUMA, Exp3 NUMA-Remote, Exp4 Locality-on-Remote)
- runtime (from `sim.out`)
- IPC (from `sim.out`)
- 5–10 relevant memory-related counters from `dumpstats.py`

Explain:

- why CC-NUMA-like topology can reduce performance for memory-bound kernels,
- why the remote configuration amplifies the effect,
- why the locality-optimized variant helps, supported by one or more statistics.